



Node

Web Programming
Lecture 14
Mahhek Khan



NODE.js

Node.js is a JavaScript runtime built on Chrome's V8 engine.

Allows you to run JavaScript on the server.

Non-blocking, event-driven architecture – great for web apps & APIs.



Express.js

Definition: Express is a minimal and flexible Node.js web application framework.

Purpose: Simplifies routing, handling HTTP requests, middleware.



Setup Node

Install Node.js: Go to nodejs.org and install the latest version.

Verify Installation: Open terminal and run:

```
node -v
```

```
npm -v
```



Setup a Project

1. Create a folder

```
mkdir my-node-app
```

```
cd my-node-app
```

2. Initialize Node.js

```
npm init -y
```

3. Install mongoose and express

```
npm install express mongoose cors dotenv
```

```
npm install --save-dev nodemon
```

```
mkdir config controllers models routes
```

4. Create main file

```
touch index.js .env
```

```
npm run dev
```



MVC

Model - Interacts with database

Views - Interacts with clients

Controller - Handle Request and Response

In MERN

Model MongoDB schema (Mongoose model)

Controller - [Node.js](#) Handles request/response

View - In MERN, React acts as the View layer (separate frontend)

Code

```
backend/  
  models/  
  controllers/  
  routes/
```

```
frontend/  
  components/  
  pages/
```

File Structure

✓ MERN-BACKEND

✓ config

JS db.js

✓ controllers

JS userController.js

✓ models

JS User.js

> node_modules

✓ routes

JS userRoutes.js

⚙ .env

JS index.js

{ } package-lock.json

{ } package.json

.env



A .env file stores environment variables used to configure an application. These variables allow us to keep configuration separate from the source code.

```
PORT=5000
```

```
MONGO_URI=mongodb://localhost:27017/mernapp
```

```
JWT_SECRET=mysecretkey
```

Keeps sensitive information (API keys, passwords) out of the code

Naming convention

Environment variables are usually written in CAPITAL LETTERS with underscores.

The .env file should NOT be committed to GitHub. It is usually added to the .gitignore file to keep sensitive data private.



run server

package.json:

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js"  
}
```

npm run dev

<http://localhost:5000>

By **convention**, developers use:

- **npm run dev** → development (nodemon, debugging, auto-reload)
- **npm start** → production (normal Node run)

index.js

```
// import express library
const express = require("express");

// import routes from another file
const itemRoutes = require("./routes/itemRoutes");

// create the express app (server)
const app = express();

// middleware to convert JSON body into JS object (req.body)
app.use(express.json());

// home route to check if server works
app.get("/", (req, res) => {
  res.send("Server is running!");
});

// example route written directly in index.js
app.get("/api/users", (req, res) => {
  const users = ["Ali", "Sara", "Ahmed"];
  res.json(users);
});

// connect external routes for items
app.use("/api/items", itemRoutes);

// define server port
const PORT = 5000;

// start the server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```



Rest Routes

Method	Path	Description
GET	<code>/items</code>	Get all items
GET	<code>/items/:id</code>	Get one item
POST	<code>/items</code>	Create item
PUT	<code>/items/:id</code>	Update item
DELETE	<code>/items/:id</code>	Delete item



What is a Route?

Route decides WHICH code should run when a request comes.

Example request:

GET /api/items

The route catches the request and sends it to the correct controller.

Example:

```
router.get("/", getItems)
```

Here:

"/" → the route path

getItems → the controller function



Connecting Routes in index.js

```
const itemRoutes = require("./routes/itemRoutes");  
app.use("/api/items", itemRoutes);
```

Meaning:

If a request starts with:

/api/items

send it to the routes file.



Routes File Example (itemRoutes.js)

```
const express = require("express");
const router = express.Router();

const { getItem, updateItem, deleteItem } = require("../controllers/itemController");

// GET all items
router.get("/", getItem); // http://localhost:5000/api/items

// UPDATE an item
router.put("/:id", updateItem); // http://localhost:5000/api/items/123

// DELETE an item
router.delete("/:id", deleteItem); // http://localhost:5000/api/items/123

module.exports = router;
```

Controller Example (itemController.js)

```
// get all items
exports.getItems = (req, res) => {
  const items = ["Laptop", "Phone", "Tablet"];
  res.json(items);
};

// update an item
exports.updateItem = (req, res) => {
  const id = req.params.id; // get id from URL
  res.send(`Item ${id} updated successfully`);
};

// delete an item
exports.deleteItem = (req, res) => {
  const id = req.params.id; // get id from URL
  res.send(`Item ${id} deleted successfully`);
};
```



Complete Request Flow

Request	Route	Controller runs
GET /api/items	<code>router.get("/")</code>	<code>getItems()</code>
PUT /api/items/123	<code>router.put("/:id")</code>	<code>updateItem()</code>
DELETE /api/items/123	<code>router.delete("/:id")</code>	<code>deleteItem()</code>



Client (Browser / Postman)

PUT /api/items/123

index.js

```
app.use('/api/items', itemRoutes)
```

Routes (itemRoutes.js)

```
router.put('/:id', updateItem)
```

Controller (itemController.js)

```
updateItem(req, res)
```

```
req.params.id = 123
```

Update Logic

Response to Client

"Item 123 updated successfully"

